

DeliveryFlow 개발 여정

작성일: 2026-08-15

1. 이 문서의 목적

이 문서는 경력 복귀 포트폴리오 프로젝트인 DeliveryFlow를 처음 준비한 시점부터 로컬 개발 환경 구축, API 구현, Docker 실행, Railway 배포까지의 과정을 정리한 학습 기록입니다. 단순히 기능 목록만 나열하지 않고, 설치와 설정에서 발생한 문제를 어떻게 이해하고 해결했는지를 남기는 데 초점을 둡니다.

민감 정보 보호를 위해 데이터베이스 비밀번호, JWT 토큰, 개인 계정 비밀번호와 같은 값은 기록하지 않습니다.

2. 프로젝트 개요

DeliveryFlow는 배송 주문 접수, 배송 기사 배정, 배송 상태 변경, 변경 이력 조회를 제공하는 Spring Boot 기반 백엔드 API입니다. 과거 배송 접수 시스템 경험을 현대적인 Java 백엔드 기술로 재구성하는 것을 목표로 했습니다.

- 소스 저장소: <https://github.com/cybdream/deliveryflow>
- 배포 API: <https://deliveryflow-production.up.railway.app>
- Swagger UI: <https://deliveryflow-production.up.railway.app/swagger-ui/index.html>
- 주요 기술: Java 17, Spring Boot, PostgreSQL, JPA, Spring Security, JWT, Docker, GitHub Actions, Railway

3. 개발 과정

3.1 프로젝트 방향과 문서화

먼저 배송 운영 관리 API를 포트폴리오 주제로 정하고, 한글과 영문 README 및 기능 명세서를 작성했습니다. 기능을 바로 만들기보다 주문, 기사, 배송, 상태 이력이라는 업무 개념과 역할별 권한을 먼저 정리했습니다. 이후 구현이 진행될 때마다 API, 인증, 오류 응답, Docker, CI, Railway 배포 문서를 추가했습니다.

이 과정에서 얻은 원칙은 다음과 같습니다.

- 기능 구현 전 API의 입력값, 응답값, 권한, 상태 전이를 문서로 정한다.
- 민감한 비밀번호와 토큰은 소스 저장소 및 공개 문서에 넣지 않는다.
- 배포 후에는 Swagger를 이용해 외부 환경에서 실제 동작을 다시 확인한다.

3.2 로컬 개발 환경 설치

Windows 환경에서 다음 도구를 설치하고 연결했습니다.

1. Eclipse Temurin 기반 JDK 17을 설치했습니다. Spring Boot 프로젝트 실행에 필요한 Java 개발 환경이며, 설치 경로는 기본값을 사용했습니다.

2. IntelliJ IDEA Community Edition 2025.0.2를 설치했습니다. Community Edition이 최신 독립 설치 프로그램으로 제공되지 않는 배포 방식으로 바뀐 뒤였으므로, JetBrains Other Versions 페이지에서 해당 버전을 내려받았습니다. 이 버전에서 Spring Boot 프로젝트 열기, Gradle 실행, 코드 작성과 디버깅이 가능함을 확인했습니다.

3. Spring Initializr에서 Gradle, Java 17, Spring Web, Spring Data JPA, PostgreSQL Driver 등 필요한 의존성을 선택하여 프로젝트를 생성했습니다.

4. PostgreSQL을 설치하고 로컬 데이터베이스를 생성했습니다.

처음 실행할 때 DataSource URL과 드라이버를 찾지 못해 애플리케이션이 시작되지 않았습니다. 이는 JPA 의존성은 있지만 데이터베이스 연결 정보가 없었기 때문이었습니다. 로컬 전용 설정 파일에 PostgreSQL 접속 정보와 활성 프로필을 설정한 뒤 정상 기동을 확인했습니다.

3.3 Git과 GitHub 연결

로컬 프로젝트 폴더를 만들고 Git 저장소를 초기화했습니다. README와 명세서부터 첫 커밋으로 관리했으며, GitHub 원격 저장소를 연결해 main 브랜치로 올렸습니다.

초기에 src refspec main does not match any와 upstream 관련 오류가 발생했습니다. 이는 첫 커밋이 없거나 로컬 브랜치와 원격 추적 브랜치가 연결되지 않았을 때 발생하는 Git 메시지였습니다. 첫 커밋 후 원격 main 브랜치를 upstream으로 지정하여 해결했습니다.

줄바꿈 LF/CRLF 경고도 확인했습니다. 이는 Windows와 Git의 줄바꿈 처리 차이로, 코드와 문서의 기능 오류는 아니었습니다. 경고의 의미와 영향을 이해한 뒤 계속 작업했습니다.

3.4 핵심 API와 데이터베이스 구현

다음 업무 흐름을 API로 구현했습니다.

1. 관리자 로그인 후 JWT 발급
2. 주문 등록과 주문 목록 조회
3. 배송 기사 등록과 목록 조회
4. 주문과 기사를 연결하는 배송 배정
5. 기사 본인의 배송 목록 조회
6. 배송 상태 변경과 변경 이력 저장
7. 관리자의 전체 배송 목록과 배송 현황 대시보드 조회

데이터는 PostgreSQL의 주문, 사용자, 배송, 배송 이력 테이블에 저장됩니다. 내부 식별자인 id는 데이터베이스가 생성하고, 사용자에게 보이는 주문번호는 날짜와 난수를 조합하여 만듭니다. 이 두 값을 분리함으로써 나중에 주문번호 정책이 바뀌어도 기존 데이터의 내부 참조를 유지할 수 있게 설계했습니다.

배송 상태는 RECEIVED, ASSIGNED, IN_DELIVERY, ON_HOLD, DELIVERED, CANCELLED로 관리합니다. 완료와 취소는 최종 상태이며, 보류와 취소에는 사유를 필수로 받습니다. 상태를 바꿀 때마다 이전 상태, 새 상태, 처리자, 시각, 사유를 배송 이력에 남깁니다.

3.5 인증과 역할별 권한

로그인 API는 관리자와 기사용 JWT를 발급합니다. Spring Security와 JWT를 적용하여 보호된 API에는 인증을 요구했습니다.

- 관리자: 주문, 기사, 배송 배정과 전체 조회, 모든 상태 변경, 이력과 대시보드 조회

- 기사: 본인에게 배정된 배송 목록 조회, 배송 시작, 완료, 보류 상태 변경

개발 중 AI 도구가 오래된 Spring Security 방식이나 Python/Flask 예시를 제안하는 경우가 있었습니다. 이를 그대로 복사하지 않고 현재 프로젝트의 Java, Spring Boot, Spring Security 버전과 사용자 엔티티 구조에 맞는 지 검토했습니다. 특히 기사 계정이 ON_HOLD 상태에서 임의로 ASSIGNED 상태로 되돌릴 수 있던 권한 문제를 확인하고, 기사는 IN_DELIVERY, DELIVERED, ON_HOLD로만 바꿀 수 있도록 검증 규칙과 테스트를 추가했습니다.

3.6 API 테스트와 Swagger

PowerShell의 REST 요청과 Swagger UI를 함께 사용해 API를 검증했습니다. 로그인 응답에서 받은 JWT는 PowerShell 변수에 잠시 보관해 Authorization 헤더에 넣어 사용했습니다. 변수는 같은 PowerShell 창이 열려 있는 동안에만 유지됨을 확인했습니다.

테스트 과정에서 다음 내용을 익혔습니다.

- 201 Created는 새 리소스가 성공적으로 생성되었음을 뜻한다.
- 403 Forbidden은 인증은 되었지만 역할 권한이 부족한 경우에 발생한다.
- 400 Bad Request는 잘못된 요청값 또는 상태 전이 규칙 위반에 사용한다.
- 목록 조회의 정렬 값은 실제 엔티티 필드 형식에 맞춰 전달해야 한다.
- Swagger의 상태 변경 입력은 자유 텍스트보다 선택 목록이 안전하므로, PATCH API를 query parameter enum 방식으로 바꿨다.

기사 배송 목록은 별도 화면이 아니라 GET /api/v1/deliveries/me API가 담당합니다. 이후 통계 요구가 생기면 기사 전용 대시보드 API를 추가하는 방향으로 남겨 두었습니다.

3.7 Docker와 WSL 2

로컬 실행 환경을 재현하기 위해 Docker Desktop을 설치하고 WSL 2 기반 Linux 컨테이너 환경을 준비했습니다. Windows에서 Linux 컨테이너를 실행하려면 Linux 커널이 필요하므로 Docker Desktop은 WSL 2를 사용합니다.

Docker Compose로 애플리케이션 컨테이너와 PostgreSQL 컨테이너를 함께 실행했습니다. 처음에는 Docker Desktop 데몬이 실행되지 않아 named pipe 연결 오류가 발생했습니다. 재부팅 후 Docker Desktop이 정상 기동된 것을 확인하고 다시 실행했습니다.

컨테이너 로그에서 Tomcat이 8080 포트로 시작되고 Spring Boot 애플리케이션이 Started 상태가 되는 것을 확인했습니다. Docker 이미지는 애플리케이션 실행에 필요한 파일과 의존성을 묶은 패키지이며, 가상머신처럼 전체 운영체제를 포함하지 않고 호스트의 Linux 커널을 공유한다는 차이도 함께 학습했습니다.

3.8 테스트와 CI

단위 테스트와 API 통합 테스트를 작성했고, GitHub Actions 워크플로를 추가했습니다. GitHub에 push할 때마다 Gradle 테스트가 실행되도록 구성했습니다.

이 단계에서 단순히 로컬에서만 실행되는 프로젝트가 아니라, 변경 사항이 원격 저장소에 반영될 때 자동으로 검증되는 기본적인 CI 흐름을 갖추게 되었습니다.

3.9 Railway 배포와 운영 환경 설정

GitHub 저장소를 Railway 서비스에 연결하고 PostgreSQL 서비스를 같은 Railway 프로젝트에 생성했습니다. 배포 과정에서 다음 문제를 해결했습니다.

1. 데이터베이스 연결 실패와 Hibernate Dialect 오류

- 원인: 앱이 JDBC URL을 만들지 못해 데이터베이스 메타데이터를 읽을 수 없었습니다.
- 해결: 앱 서비스의 Spring 데이터소스 환경 변수가 PostgreSQL 서비스의 내부 변수(PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD)를 참조하도록 설정했습니다.

2. 포트 설정 확인

- Railway가 제공하는 PORT 환경 변수를 Spring Boot가 사용하도록 설정하고, 컨테이너가 해당 포트에서 정상 기동되는지 로그로 확인했습니다.

3. 외부 Swagger 호출 오류

- 배포 도메인에서 Swagger를 열고 API를 호출할 때 CORS 또는 URL scheme 오류가 보인 적이 있었습니다.
- 앱의 공개 도메인과 프록시 환경 설정을 확인한 뒤, Railway 배포 URL에서 Swagger UI와 API 호출이 정상 동작하는 것을 검증했습니다.

최종적으로 Railway에서 서비스가 Online 상태가 되고, 로그인, 주문 등록, 주문 조회를 외부 URL에서 정상 실행했습니다.

4. 현재 구현 상태

완료된 기능은 주문 관리, 기사 관리, 배송 배정, 배송 상태 전이와 이력, JWT 인증과 역할 권한, 배송 목록 검색, 오류 응답 다국어 처리, 대시보드, Swagger, Docker, GitHub Actions CI, Railway 배포입니다.

공개 README에는 테스트 계정의 이메일만 안내하고 비밀번호는 기록하지 않습니다. 이 정책은 배포된 데이터가 임의로 변경되는 것을 줄이고, 자격 증명이 남는 것을 막기 위한 것입니다.

5. 다음 작업

다음 작업은 우선순위에 따라 진행합니다.

1. Railway Swagger에서 관리자 주문 등록 - 기사 배정 - 기사 배송 시작/완료 - 관리자 이력/대시보드 확인의 전체 흐름을 다시 검증합니다.
2. 검증 화면과 결과를 포트폴리오 증빙 자료로 정리합니다.
3. 기사별 당일 배송 건수와 완료율 통계, 고객용 주문번호 배송 조회, 테스트 확장 및 코드 리팩터링 중 하나를 다음 기능으로 선택합니다.

6. 회고

이번 프로젝트를 통해 Java와 Spring Boot 개발 환경을 다시 구축하고, 데이터베이스 연결, 인증, REST API, Docker 컨테이너, CI, 클라우드 배포까지 하나의 흐름으로 경험했습니다. 오류가 발생했을 때 메시지를 그대로 해결책으로 받아들이기보다, 현재 환경과 의존성 버전, 권한 규칙, 서비스 간 연결 구조를 확인하는 방식으로 접근했습니다.

이 문서는 이후 기능을 추가하거나 면접 준비를 할 때, 무엇을 만들었는지뿐 아니라 어떤 문제를 해결하며 프로젝트를 완성했는지 설명하는 기반 자료로 사용합니다.